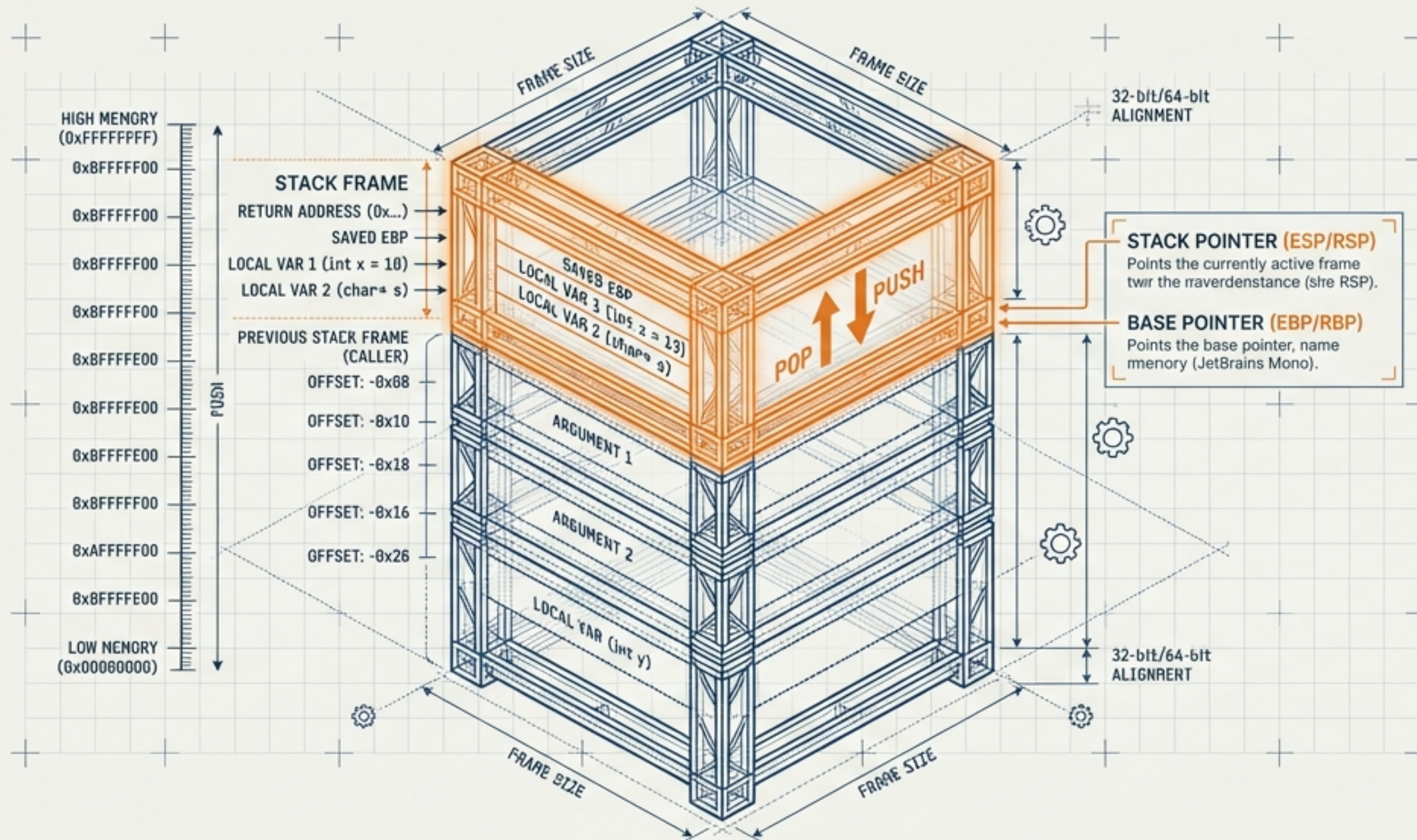


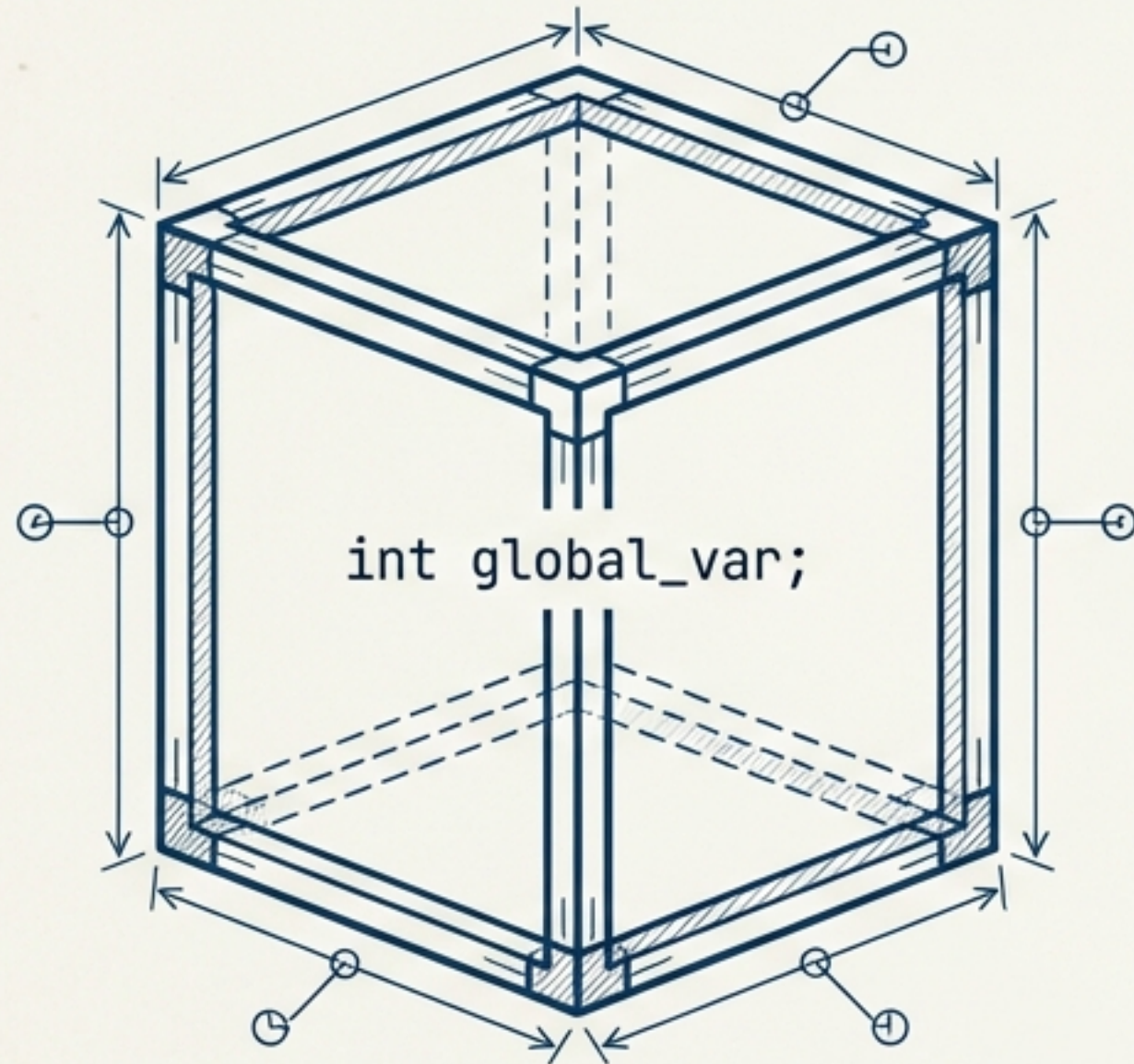
THE STACK: LOCAL VARIABLES & MEMORY ALLOCATION

A mechanical deconstruction of stack frames, offsets, and assembly internals.



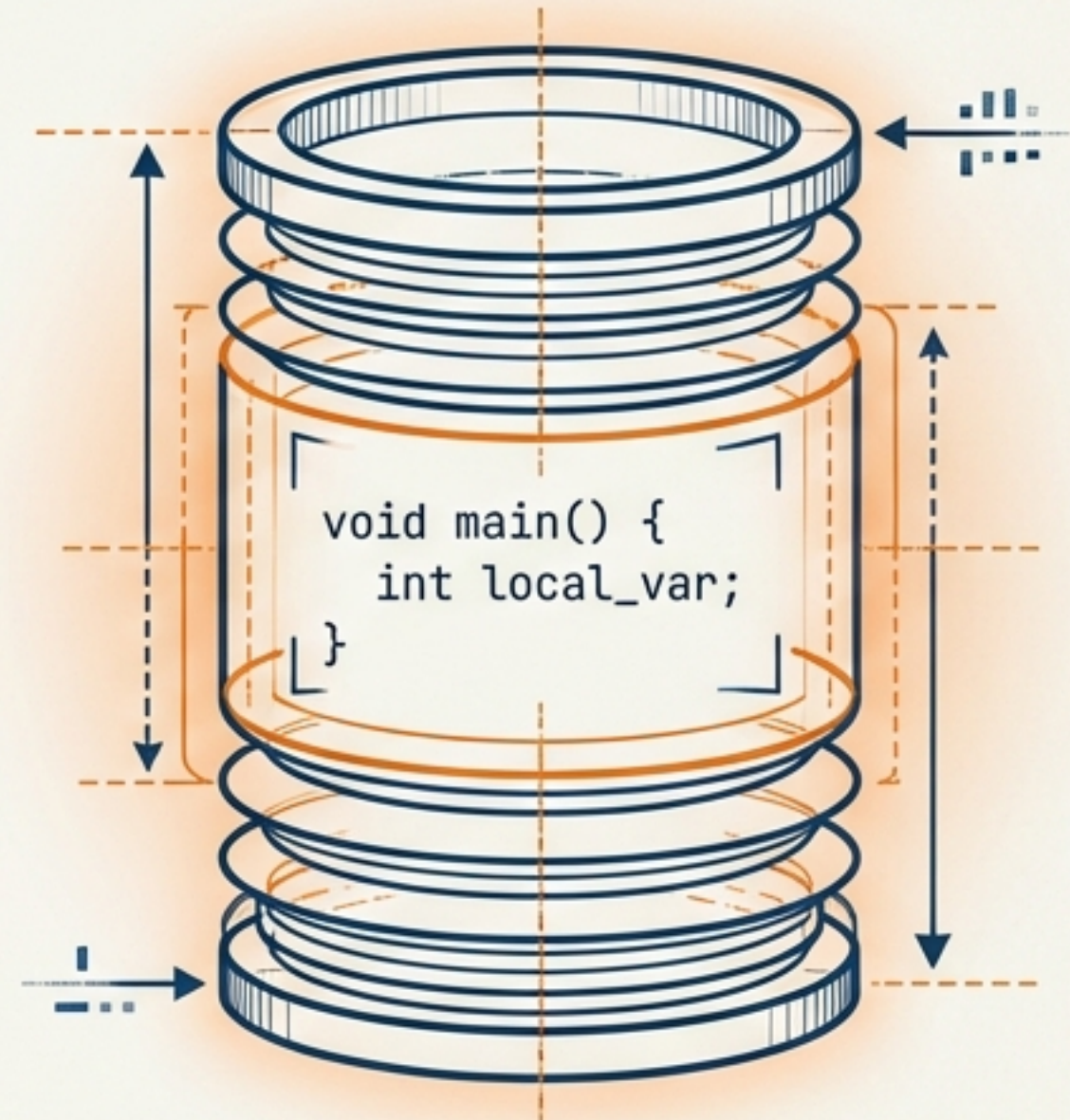
SCOPE DICTATES STORAGE

DATA SEGMENT



Fixed Size. Permanent Lifetime. Global Scope.

THE STACK

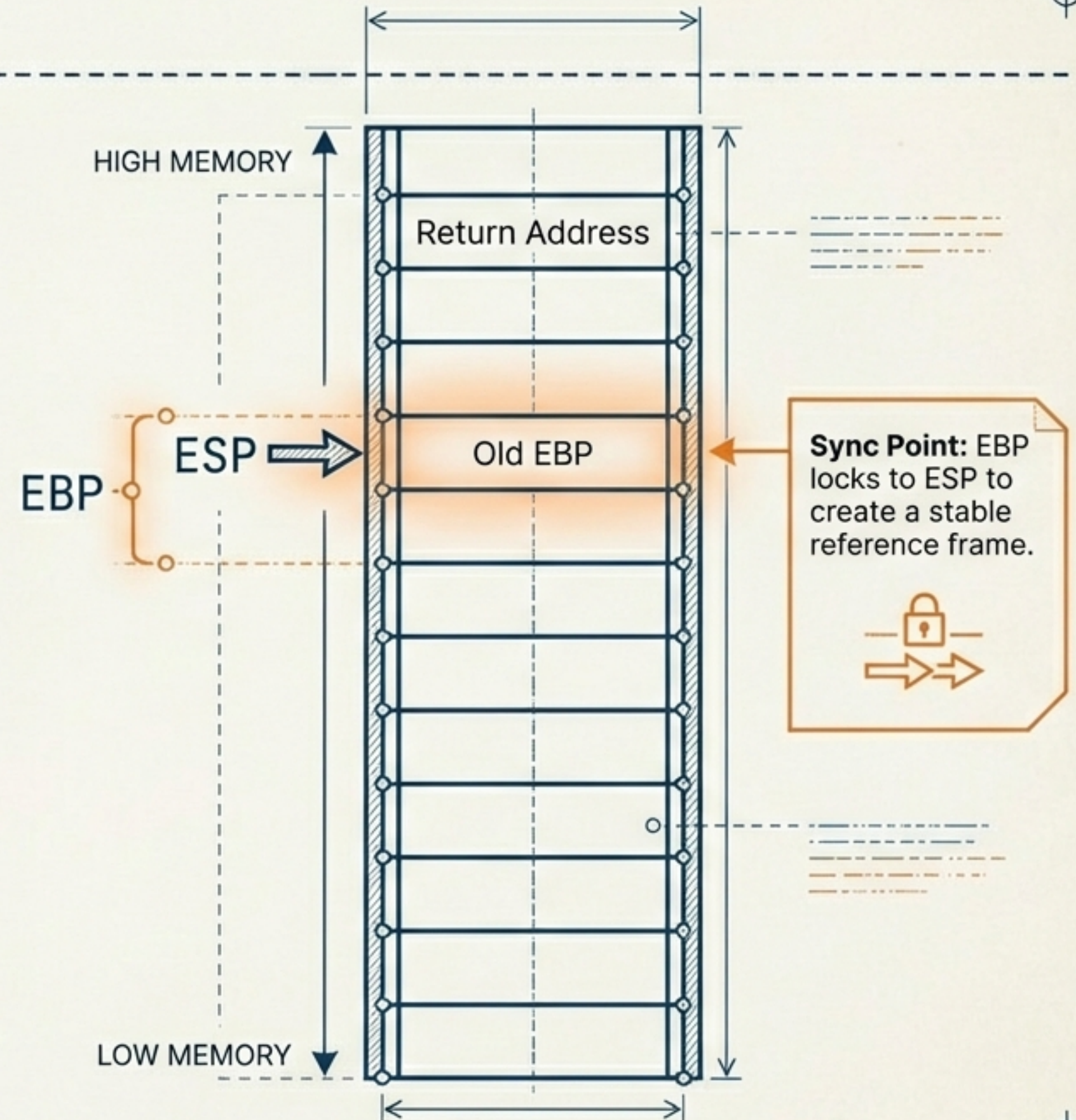


Dynamic Size. Temporary Lifetime. Function Scope.

Local variables are **stack-based**. We use the stack's address space to temporarily allocate storage, contrasting with global variables defined outside functions.

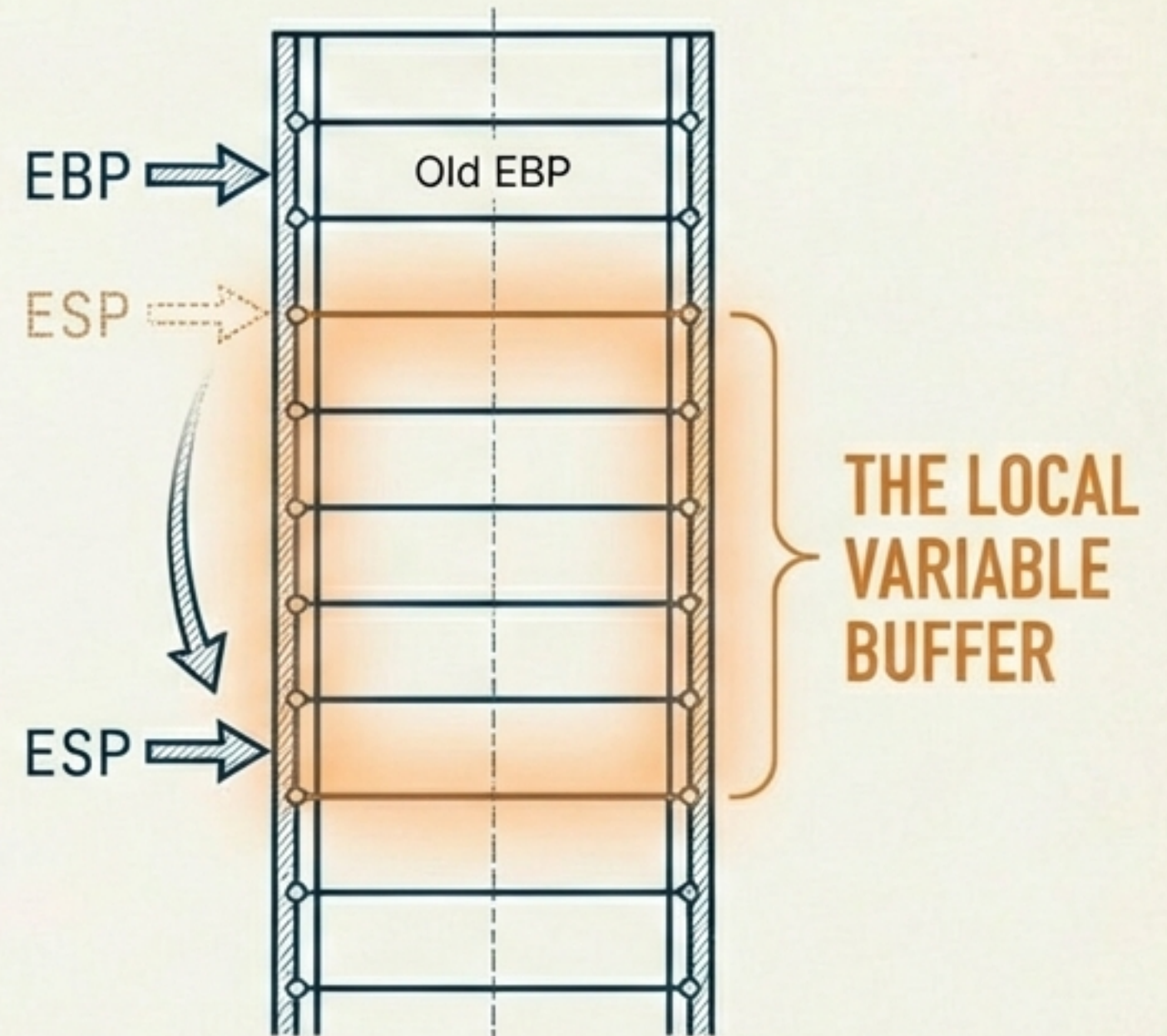
PHASE 1: THE PROLOGUE (THE ANCHOR)

```
PUSH EBP  
MOV EBP, ESP
```



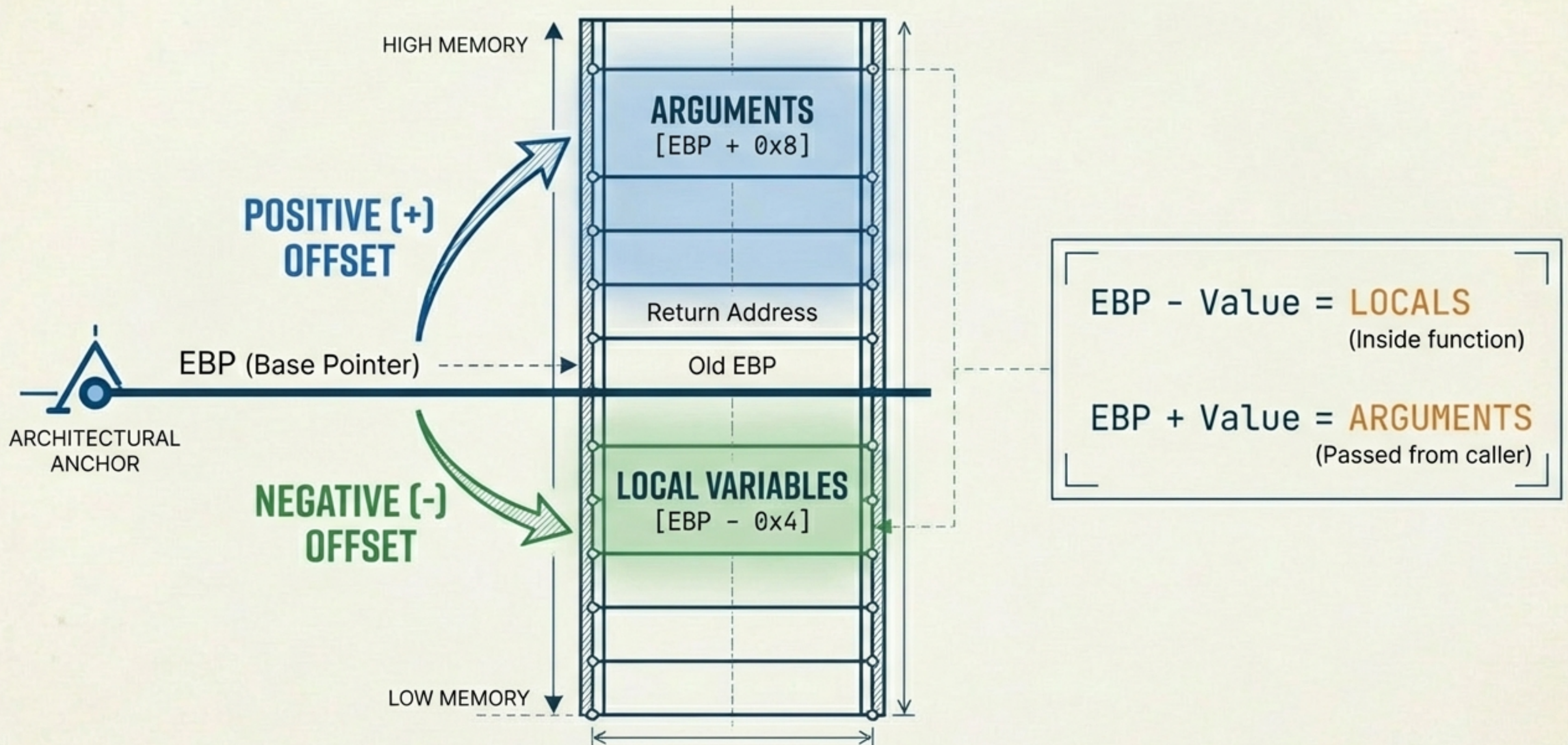
PHASE 2: ALLOCATION VIA SUBTRACTION

```
SUB ESP, 0xC  
; Subtract 12 bytes
```



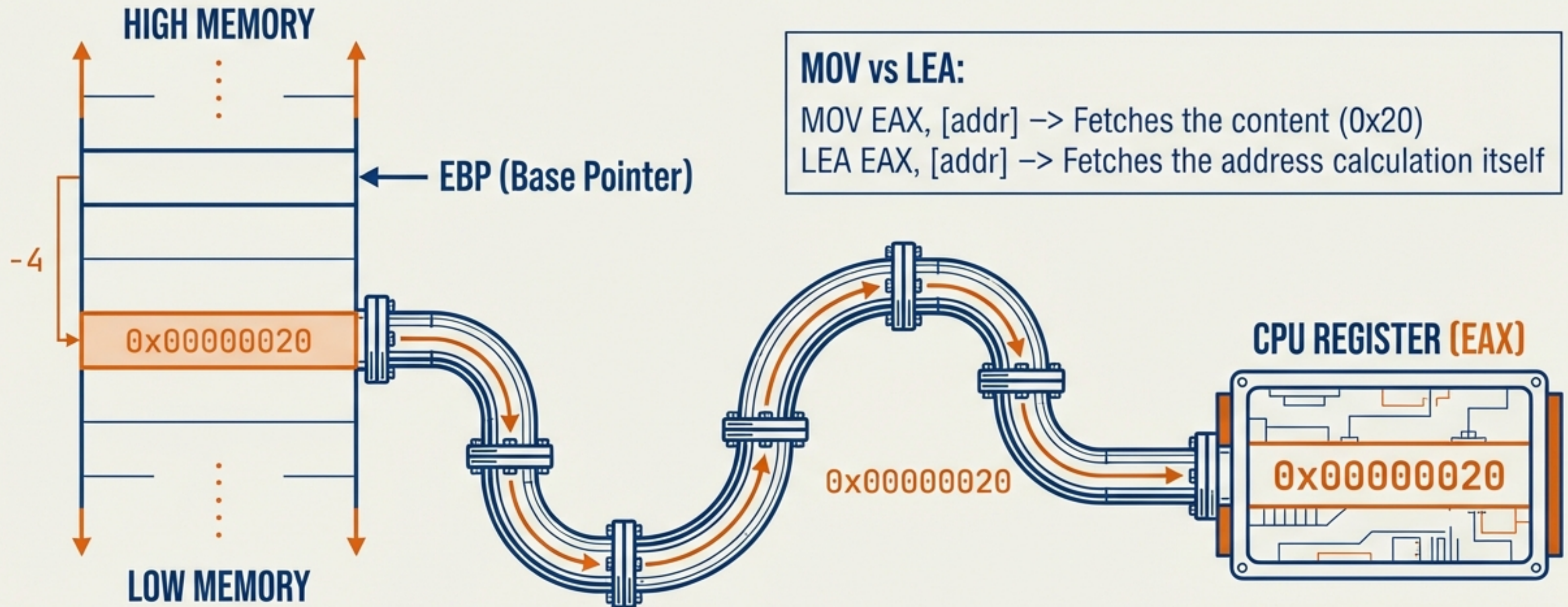
The Stack grows **DOWN** (High -> Low).
Subtracting from the pointer physically
opens a gap for data.

THE NAVIGATION MAP: EBP OFFSETS



DEREFERENCING: FETCHING THE DATA

```
MOV EAX, [EBP - 4]
```

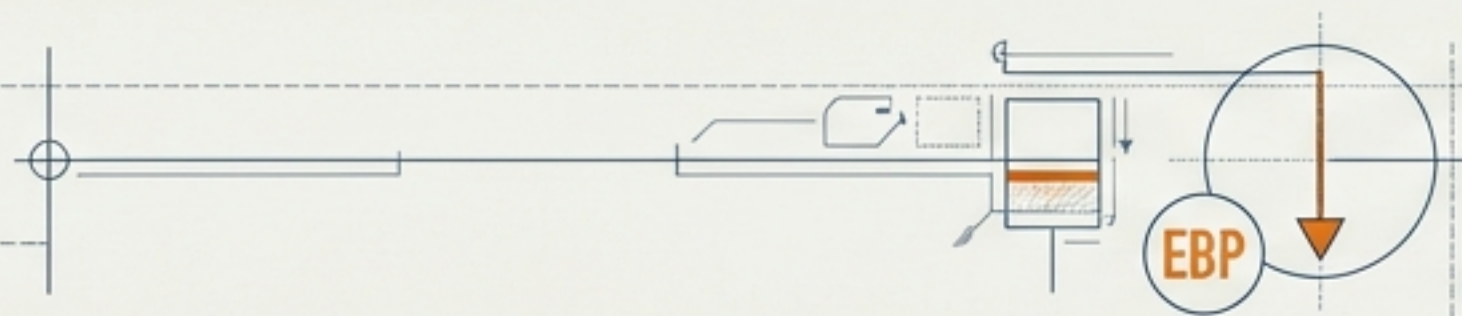


THE SHORTCUTS: ENTER & LEAVE

ENTER (Allocation)

{ = **PUSH** EBP + **MOV** EBP, ESP + **SUB** ESP, X }

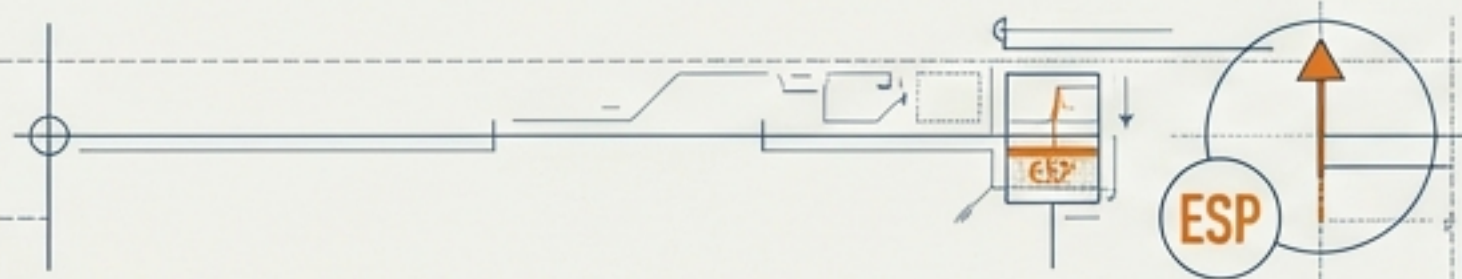
Rare. Slow. Often skipped by modern compilers.



LEAVE (Cleanup)

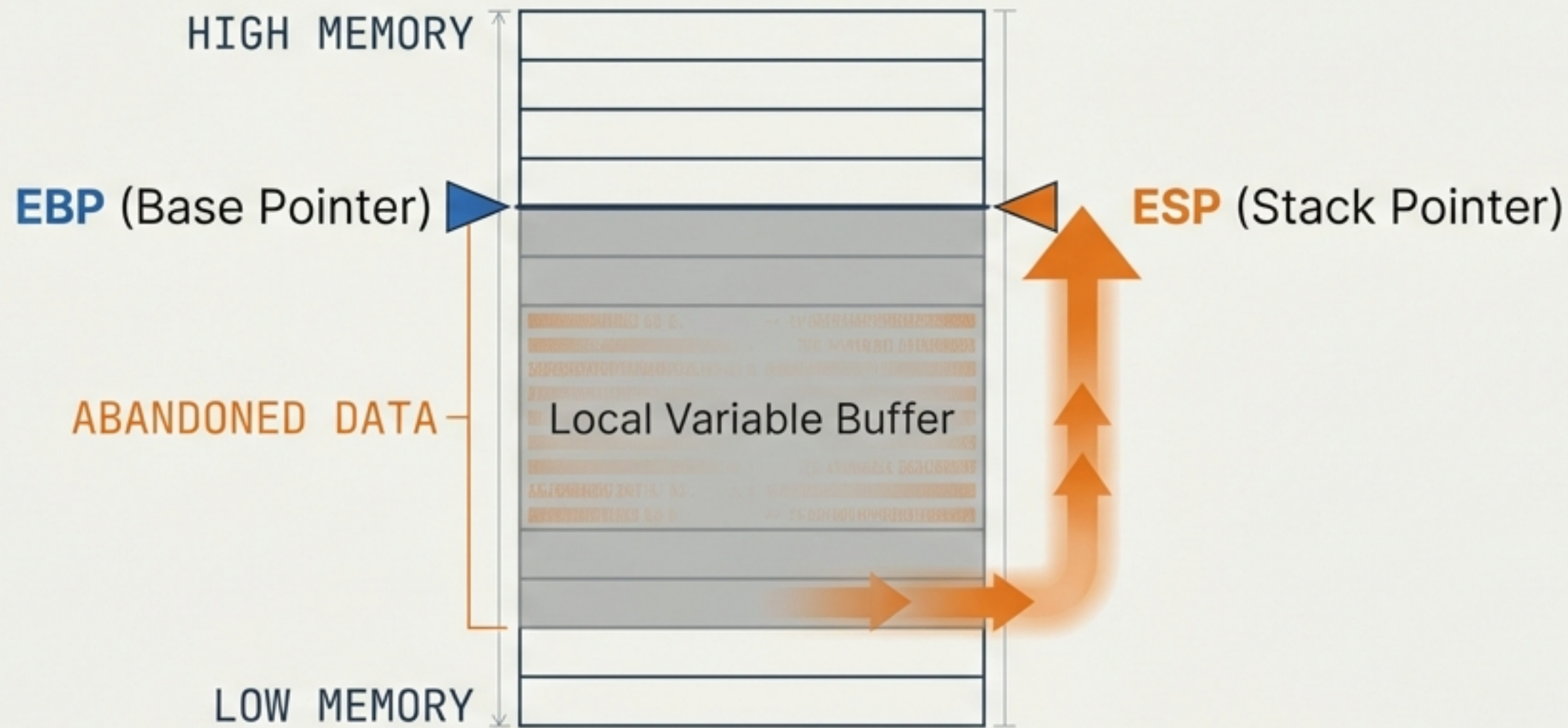
{ = **MOV** ESP, EBP + **POP** EBP }

Common. Efficient. Standard in function epilogues.



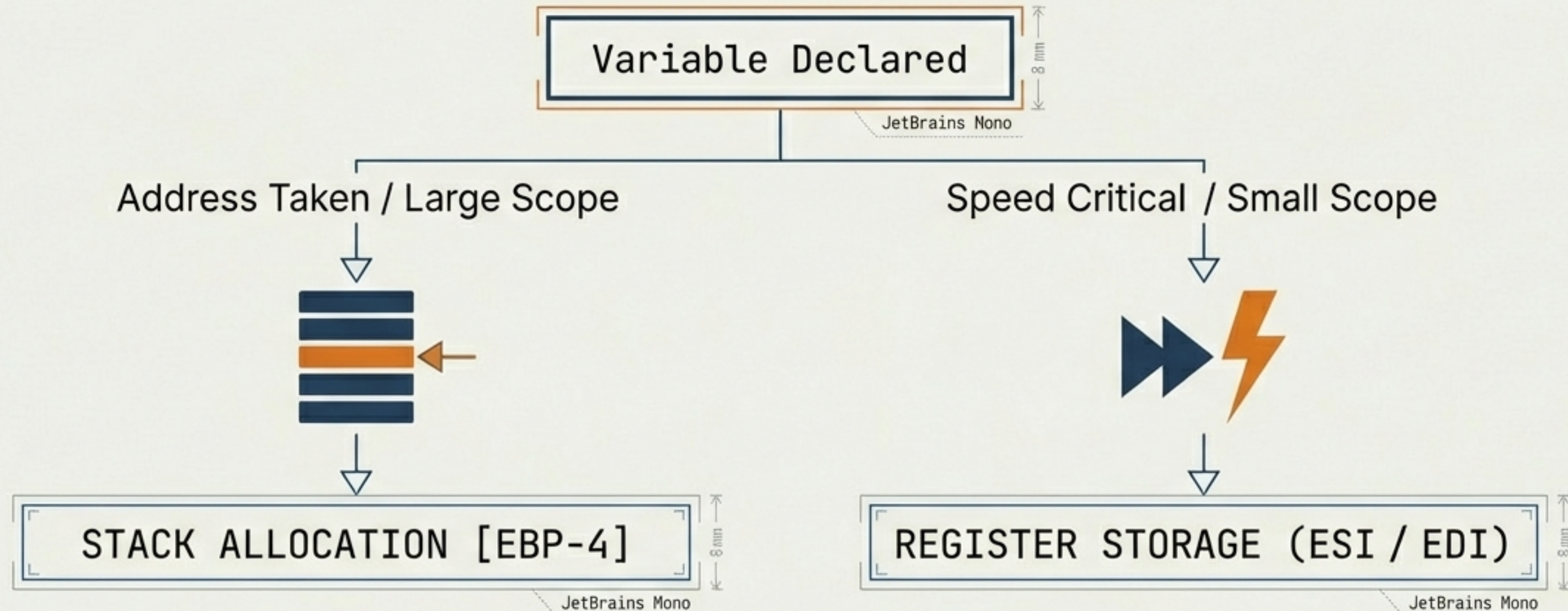
PHASE 3: THE EPILOGUE (THE SNAP BACK)

```
MOV ESP, EBP
```



Deallocation is not erasure. It is simply moving the pointer.
The data remains in memory until overwritten.

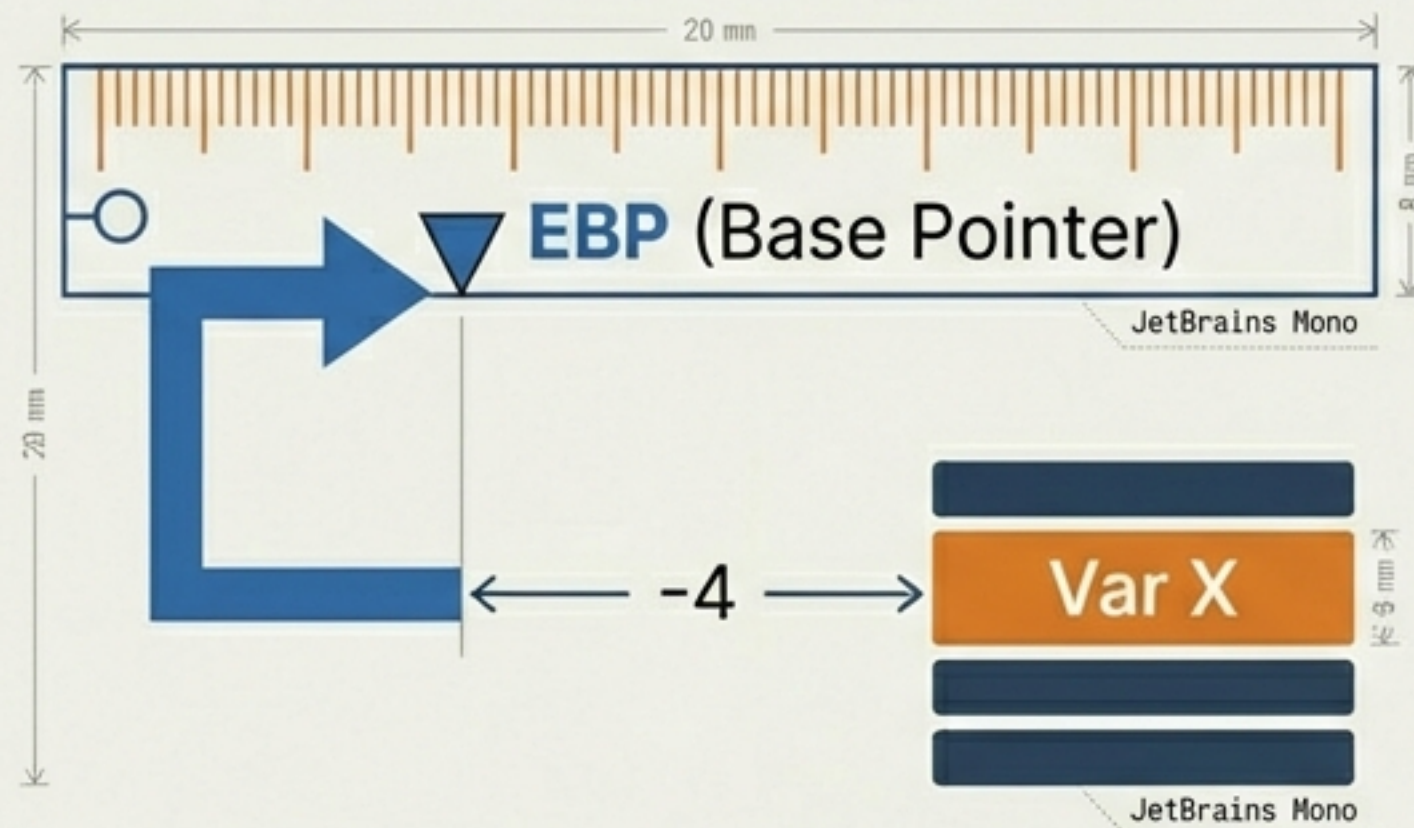
COMPILER REALITY: STACK VS. REGISTER



WARNING: Optimizing compilers may skip the stack entirely, storing local variables directly in CPU registers for performance.

THE MOVING TARGET: ESP-BASED ADDRESSING

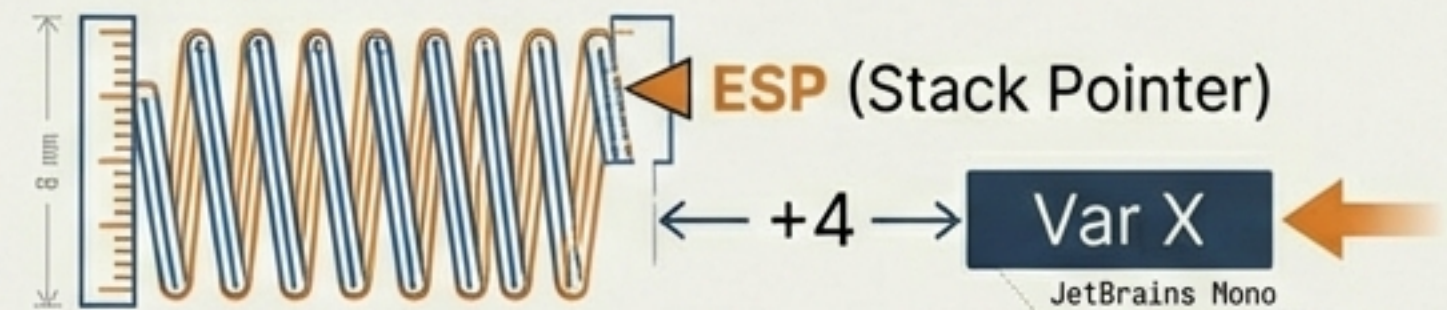
EBP Addressing (Stable)



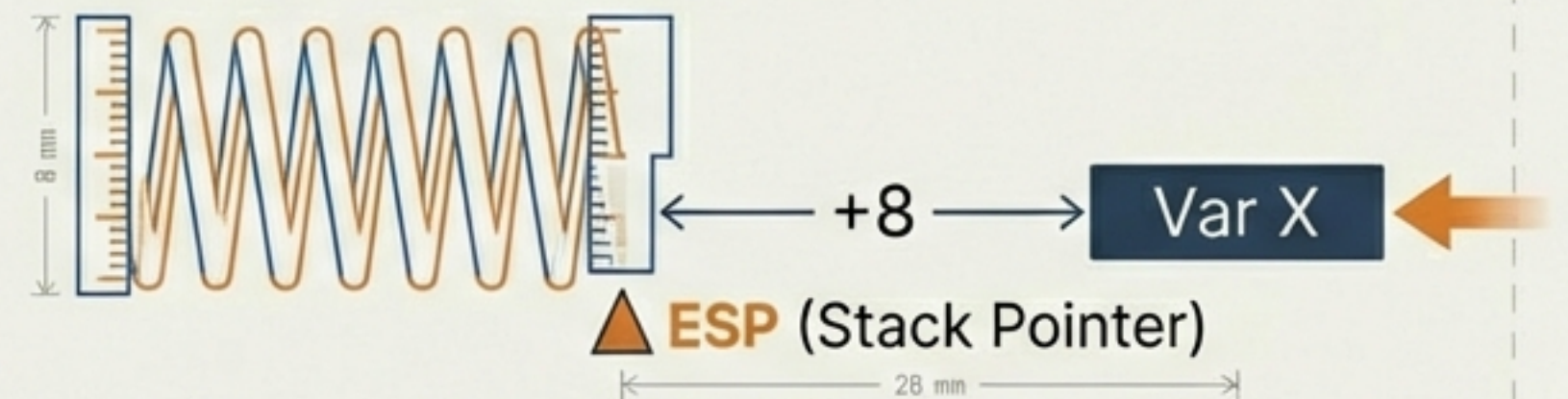
Offset remains constant (-4) regardless of stack activity.

ESP Addressing (Volatile)

State 1:



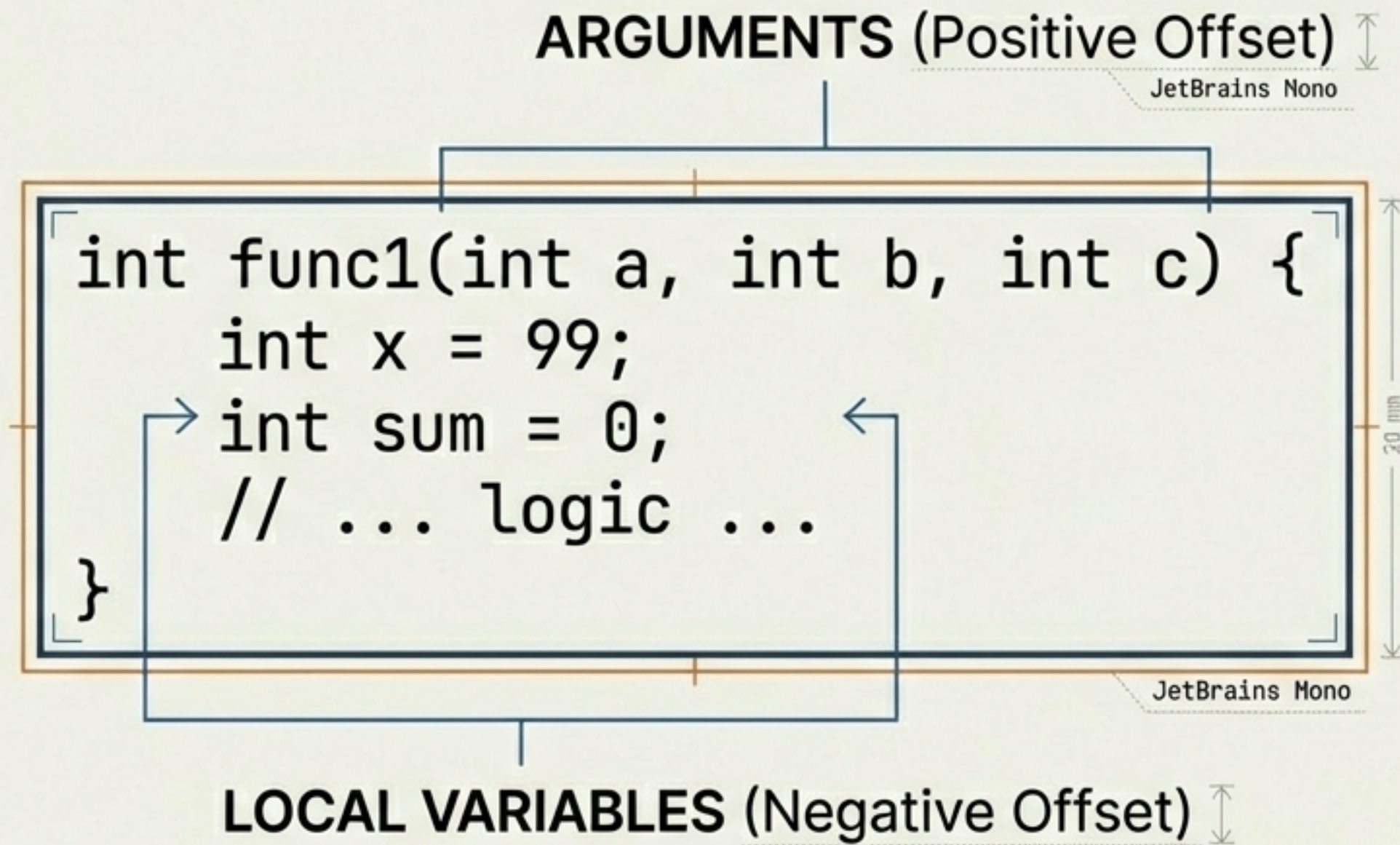
State 2 (After a Push):



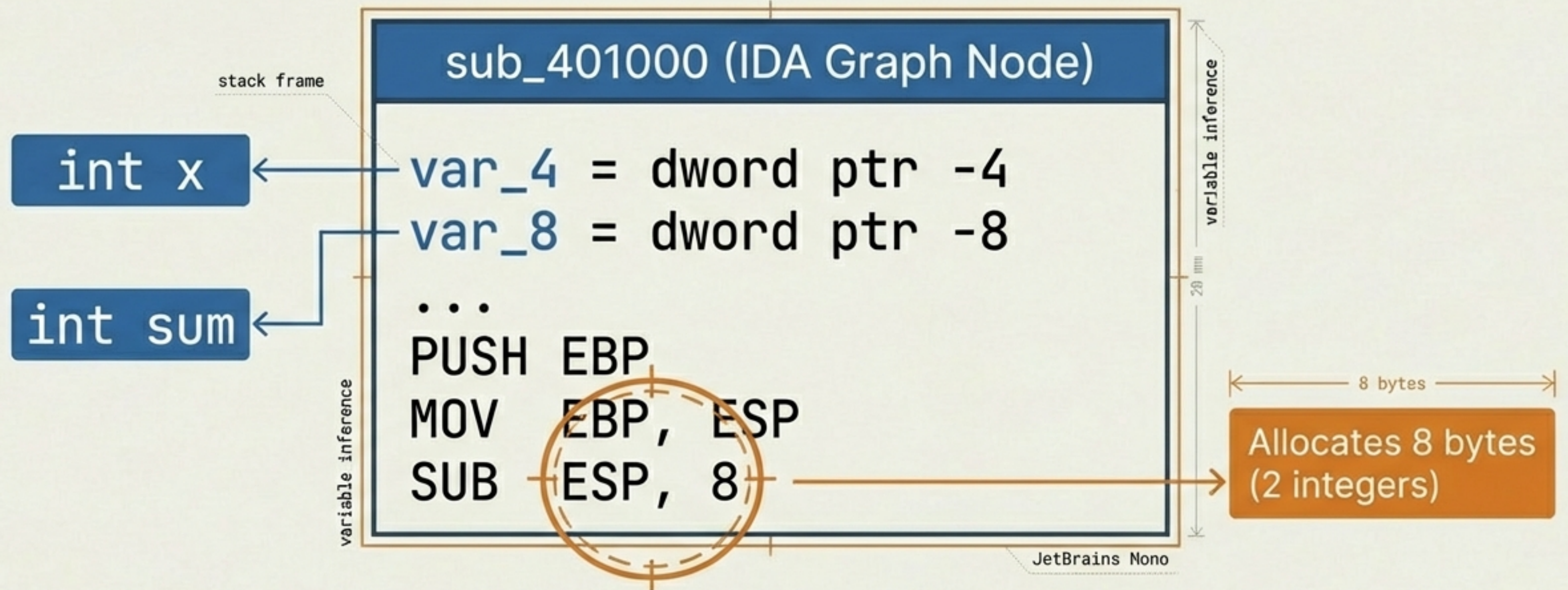
Offset **SHIFTS** as the stack grows.
Harder for humans to read.

CASE STUDY: THE SOURCE CODE

Objective: Watch how 'sum' and 'x' are physically mapped in memory.

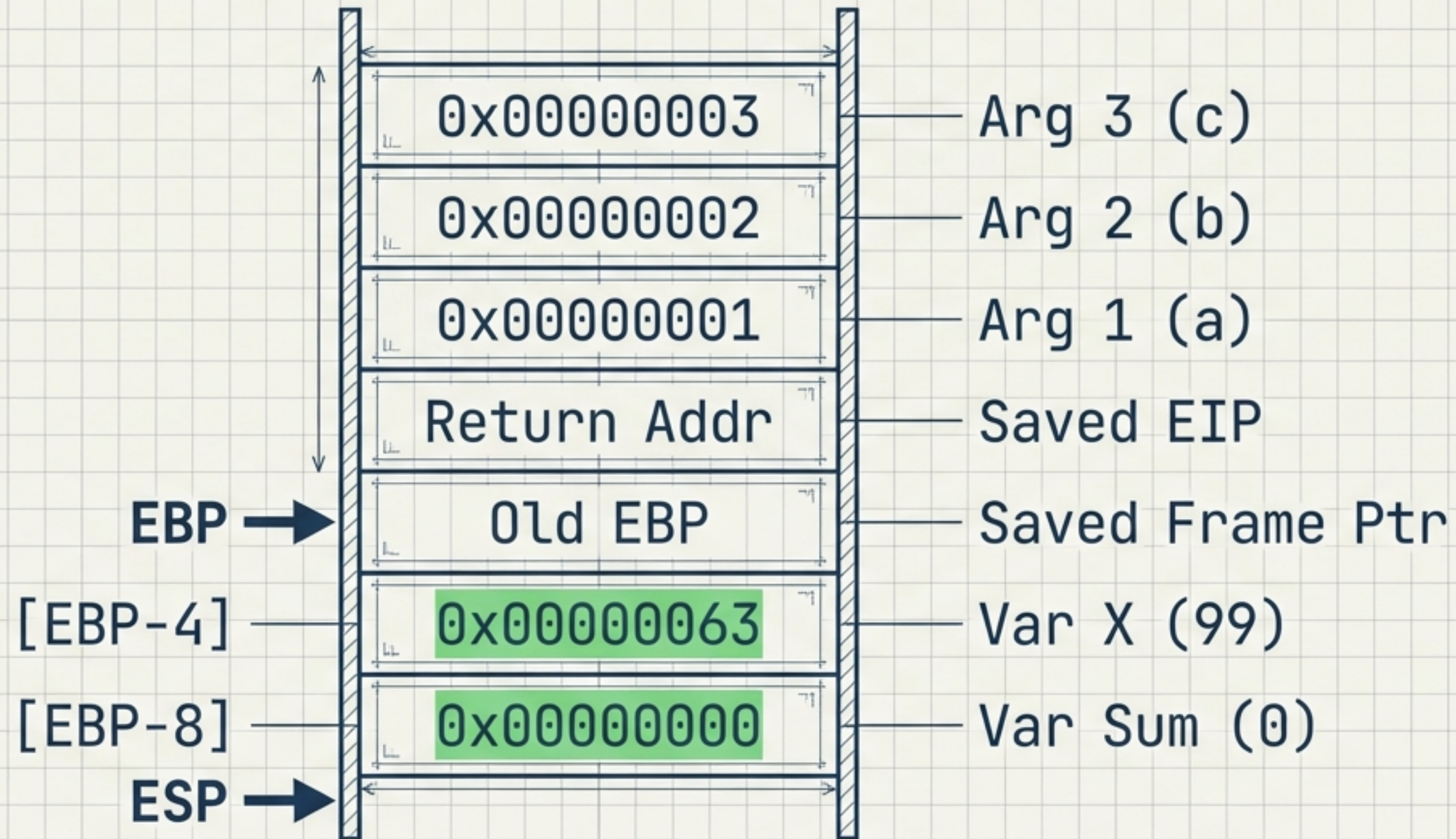


CASE STUDY: THE DISASSEMBLY (IDA VIEW)



The disassembler infers variable names (**var_4**) based on the stack subtraction.

CASE STUDY: THE POPULATED STACK



ESSENTIAL MECHANICS

- 1. LOCALS = NEGATIVE OFFSETS**
(EBP minus X)
- 2. ARGUMENTS = POSITIVE OFFSETS**
(EBP plus X)
- 3. ALLOCATION = SUBTRACTION**
SUB ESP, Value grows the stack down
- 4. DEALLOCATION = RESTORATION**
MOV ESP, EBP abandons the data
- 5. SCOPE = STORAGE**
Locals are stack-based, Globals are not

FINAL CHECK

```
MOV [EBP - 0x10], EAX
```

Are we writing to an Argument or a Local Variable?

ANSWER: LOCAL VARIABLE

Reason: The offset is NEGATIVE (-). We are reaching down into the stack frame's allocated buffer.